

### Actividad 3: Estructuras de datos

#### 1. Archivo: Recursividad.java

##### Objetivo del programa

Este programa calcula la suma de los dígitos de tres números de forma repetida hasta reducirlos a un solo dígito. Con esto, logramos obtener una clave o código de acceso.

##### Análisis del código

El archivo cuenta con dos métodos recursivos principales:

##### A. sumaDigitos(int n)

Aquí sumamos los dígitos individuales de un número entero n:

```
public static int sumaDigitos(int n) {  
    if (n == 0) {  
        return 0; // Caso base: si ya no quedan dígitos por sumar  
    }  
    return n % 10 + sumaDigitos(n / 10); // Caso recursivo  
}
```

Caso base: Cuando n es igual a 0, devolvemos 0. Esto es lo que detiene la recursión cuando ya terminamos de procesar todos los dígitos.

Caso recursivo:

$n \% 10$ : Extrae el último dígito del número (por ejemplo,  $84729 \% 10$  es igual a 9).

$n / 10$ : Elimina el último dígito mediante división entera (por ejemplo,  $84729 / 10$  nos da 8472).

La llamada recursiva simplemente suma el dígito actual con la suma de los dígitos que van quedando.

B. `sumaDigitosRepetida(int n)`

Este método se asegura de que el resultado final siempre sea un único dígito:

Java

```
public static int sumaDigitosRepetida(int n) {  
    if (n < 10) {  
        return n; // Caso base: ya es de un solo dígito  
    }  
    return sumaDigitosRepetida(sumaDigitos(n)); // Caso recursivo  
}
```

Caso base: Si el número ya es menor a 10, no necesitamos reducirlo más y lo devolvemos directamente.

Caso recursivo: Si la suma aún tiene dos o más dígitos (por ejemplo, si nos dio 30), volvemos a invocar el método pasándole como argumento el resultado anterior.

Ejecución principal (main)

Calculamos la clave evaluando los tres números que nos dieron:

1. 84729:  $8+4+7+2+9$  da 30, y luego  $3+0$  nos da 3.
2. 56318:  $5+6+3+1+8$  da 23, y luego  $2+3$  nos da 5.
3. 92746:  $9+2+7+4+6$  da 28, luego  $2+8$  da 10, y finalmente  $1+0$  nos da 1.

El resultado final que veremos en consola es: La Clave es: 3 5 1.

## 2. Archivo: DivideyVenceras.java

Objetivo del programa

Aquí buscamos encontrar el valor máximo dentro de un arreglo desordenado. Para lograrlo, aplicamos la técnica de Divide y Vencerás, partiendo el problema a la mitad en cada paso de la recursión.

Análisis del código: Método `max(int[] array, int start, int end)`

```

static int max(int[] array, int start, int end) {
    if (start == end) {
        return array[start]; // Caso base: subarreglo de 1 solo elemento
    }

    int middle = (start + end) / 2;

    int leftMax = max(array, start, middle);    // Conquistar mitad
    izquierda

    int rightMax = max(array, middle + 1, end); // Conquistar mitad
    derecha

    return Math.max(leftMax, rightMax);        // Combinar soluciones
}

```

Aplicamos las 3 fases de Divide y Vencerás de esta manera:

### 1. Dividir:

Calculamos el índice medio del rango actual partiendo el arreglo en dos mitades: la mitad izquierda (desde start hasta middle) y la mitad derecha (desde middle + 1 hasta end).

### 2. Conquistar (Caso base y recursión):

Como caso base, cuando el rango contiene un solo elemento, el máximo es ese mismo número. Luego, resolvemos ambas mitades de forma independiente haciendo llamadas recursivas.

### 3. Combinar:

Comparamos los dos máximos que obtuvimos y devolvemos el mayor de ambos hacia arriba en nuestro árbol de llamadas.

### Complejidad

Temporal:  $n$ , ya que necesitamos visitar cada elemento en las hojas de nuestro árbol de recursión.

Espacial:  $O(\log n)$  debido a la memoria que requiere la pila de llamadas en el sistema.

### 3. Archivo: Backtracking.java (Explicación del método buscar)

En este archivo, nuestro objetivo es guiar a un agente a través de un laberinto desde el punto de inicio S hasta la meta T. Durante el proceso, registramos cada celda que visitamos para encontrar la ruta más corta posible.

### Código del método buscar

```
static void buscar(int r, int c, int tr, int tc, List<int[]> camino) {  
  
    // PASO 1. Si (r,c) es la meta:  
    if (r == tr && c == tc) {  
        if (mejorCamino == null || camino.size() < mejorCamino.size()) {  
            mejorCamino = new ArrayList<>(camino);  
        }  
    }  
}
```

```

    }
    return;
}
if (mejorCamino != null && camino.size() + Math.abs(r - tr) +
Math.abs(c - tc) >= mejorCamino.size()) {
    return;
}

```

// PASO 2. Prueba cada una de las 4 direcciones usando dr[] y dc[]:

```
for (int i = 0; i < 4; i++) {
```

```
    int nr = r + dr[i];
```

```
    int nc = c + dc[i];
```

```
    // Validar límites del laberinto
```

```
    if (nr < 0 || nr >= filas || nc < 0 || nc >= columnas) {
```

```
        continue;
```

```
    }
```

```
    // Validar si es pared "#" o ya visitado "V"
```

```
    if (laberinto[nr][nc] == "#" || laberinto[nr][nc] == "V") {
```

```
        continue;
```

```
    }
```

PASO 3. Moverse (Backtracking):

```
char original = laberinto[nr][nc];
```

```

    laberinto[nr][nc] = "V";           // 1. Marcar como visitado

    camino.add(new int[] { nr, nc });   // 2. Agregar al camino actual
    System.out.println(nr + "," + nc);  // 3. Imprimir celda explorada

    buscar(nr, nc, tr, tc, camino);     // 4. Exploración recursiva

    // AL REGRESAR deshace el movimiento:
    camino.remove(camino.size() - 1);    // 5. Deshacer del camino
    laberinto[nr][nc] = original;        // 6. Restaurar estado original
}
}

```

¿Cómo funciona internamente el método paso a paso?

#### 1. Caso Base (Llegada a la meta):

Primero, revisamos si nuestra posición actual coincide con las coordenadas de la meta.

Si ya llegamos, verificamos si es el primer camino que encontramos o si esta nueva ruta toma menos pasos que nuestro récord anterior.

Si resulta ser una mejor ruta, guardamos una copia independiente y regresamos para dejar que el algoritmo siga buscando otras posibles rutas más cortas.

## 2. Optimización con Poda (Branch & Bound):

En laberintos muy grandes, si intentamos explorar todos los caminos posibles nos enfrentaríamos a un crecimiento enorme ( $4^N$  llamadas), lo que congelaría el programa.

Para evitarlo, usamos la poda, que calcula la distancia restante hacia la meta.

Si los pasos que ya dimos, sumados a los pasos mínimos que nos faltan, igualan o superan la longitud del mejor camino que ya conocemos, detenemos la búsqueda por ahí de inmediato. Así nos ahorramos millones de exploraciones innecesarias.

## 3. Exploración de Direcciones y Validación:

Usamos arreglos de desplazamiento para intentar movernos en las 4 direcciones principales: arriba, abajo, izquierda y derecha.

Control de límites: Si el siguiente paso nos saca de las dimensiones del laberinto, simplemente lo descartamos.

Obstáculos y ciclos: Si nos topamos con una pared (#) o con una casilla que ya visitamos en esta ruta (V), también lo descartamos para no quedarnos atrapados en un ciclo infinito.



#### 4. El Ciclo de Backtracking (Avanzar, Explorar y Retroceder):

Avanzar: Guardamos cómo estaba la celda originalmente, la marcamos como visitada (V) para bloquearla temporalmente, la agregamos a nuestro camino y la mostramos en consola.

Recursión: Volvemos a llamar al método para seguir explorando desde esta nueva posición.

Retroceso (Backtrack): Cuando terminamos de explorar esa ruta y regresamos de la llamada, tenemos que dejar la celda exactamente como la encontramos. La quitamos de nuestro camino y restauramos su estado original. Esto es clave porque permite que otras rutas alternativas válidas puedan volver a usar esa misma celda más adelante.

#### Conclusiones

La recursividad nos ayuda a simplificar problemas matemáticos complejos al dividirlos en su forma más natural y lógica ( $n$  y  $n - 1$ ).

La técnica de Divide y Vencerás hace que nuestras búsquedas sean mucho más eficientes, ya que reduce el problema dividiéndolo en partes más pequeñas que podemos resolver y combinar fácilmente.

El Backtracking es una herramienta excelente para salir de laberintos o recorrer árboles de decisión. Eso sí, acompañarlo con una buena condición de poda es vital para asegurarnos de que el algoritmo termine su trabajo en un tiempo razonable.

